



EAST WEST UNIVERSITY

CSE103: Structured Programming
[Fall 2024]

Project Report
Event Management System

Course Code : CSE103
Course Title : Structured Programming
Instructor Name : Md. Asif Khan Rifat
Section : 29
Group Number : 07
Group Name : LOGIC LEGENDS
Date of Submission: 16-1-2025

Submitted by:

Student ID	Student Name	Contribution
2024-3-60-455	MD Junayed Ahmed Shanto	Team Lead
2024-3-60-456	Tasfia Rahman Lefa	Generated event report
2024-3-60-457	Sabiha Khanam	Added events
2024-3-60-458	Shahriar Hassan Mojumder	Created the program
2024-3-60-459	Nur Maisha Akter	Registered participants

1. Introduction

Objective and Purpose:

The objective of this project is to develop an efficient and user-friendly **Event Management System** that allows users to create, update, and delete events while maintaining participant registration and event data. It aims to simplify event organization, enhance participant management, and generate detailed reports for better planning and analysis.

Project Topic:

Event Management System for creating, managing, and reporting events and participants using C programming.

Tools and IDE:

- Programming Language: C
- IDE: Code::Blocks , Virtual Code Studio (VS Code)
- Compiler: GCC

2. Problem Statement

Problem-definition:

Managing events and participant registrations manually can be error-prone, time-consuming, and inefficient, necessitating an automated system to streamline event creation, updates, participant tracking, and reporting

Significance:

This project simplifies event management by automating participant registrations, reducing manual errors, saving time, and providing real-time access to event data, ensuring efficient coordination and decision-making.

3. Functional Requirements

i. Event Management:

- Ability to create, update, and delete event details, including name, date, location, description, maximum participants, and current registrations.

ii. Participant Management:

- Capability to register participants, store their contact details, and manage their event registrations.

- Limit the number of events a participant can register for and enforce participant caps per event.

iii. Search Functionality:

- Search for participants by name or by the events they are registered in.

iv. Reporting:

- Generate reports displaying event details, including the number of participants registered for each event.

v. Validation and Constraints:

- Enforce constraints such as valid date formats, participant limits per event, and maximum allowable registrations per participant.

User Interaction:

- Provide a simple and interactive command-line interface to navigate the system options and features efficiently.

4. Project Design

Flowchart:

A[Start] --> B[Main Menu]

B --> C[Create Event]

B --> D[Update Event]

B --> E[Delete Event]

B --> F[Register Participant]

B --> G[Search Participant by Name]

B --> H[Search Participant by Event]

B --> I[Generate Report]

B --> J[Exit]

C --> B

D --> B

E --> B

F --> B

G --> B

H --> B

I --> B

J --> K[End])

Modular Design:

The program is divided into the following functions:

1. Event Management Module

- **Purpose:** Handle creation, updating, and deletion of events.
 - **Functions:**
 - `createEvent(int n)`: Allows adding new events with attributes such as name, date, location, description, and maximum participants.
 - `updateEvent()`: Enables modification of event details (name, date, location, description).
 - `deleteEvent()`: Removes an event from the system.
-

2. Participant Management Module

- **Purpose:** Manage participants and their registrations for events.
 - **Functions:**
 - `registerParticipant(int n)`: Adds a participant and allows registration to specific events.
 - `searchParticipantByName()`: Finds participant details by their name.
 - `searchParticipantByEvent()`: Lists all participants registered for a particular event.
-

3. Report Generation Module

- **Purpose:** Generate insights and summaries about events and participants.
- **Functions:**
 - `generateReport()`: Outputs a list of all events, their details, and the number of registered participants.

5. Code Implementation

5.1 Main Function

Snippet:

```
int main()
{
    int choice, number;
    printf("Enter the number of the events you wants to manage by this code :");
    scanf("%d", &number);
    printf("===== EVENT MANAGEMENT SYSTEM =====\n\n");

    do
    {
        printf("1. Create Event\n2. Update Event\n3. Delete Event\n");
        printf("4. Register Participant\n5. Search Participant by Name\n6. Search Participant by Event\n7. Generate Report\n8. Exit\n\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                createEvent(number);
                break;
            case 2:
                updateEvent();
                break;
            case 3:
                deleteEvent();
                break;
            case 4:
                registerParticipant(number);
                break;
            case 5:
                searchParticipantByName();
                break;
            case 6:
                searchParticipantByEvent();
                break;
            case 7:
                generateReport();
                break;
            case 8:
                printf("Exiting...\n\n");
                break;
            default:
                printf("Invalid choice!\n\n");
        }
    }
    while (choice != 8);
    return 0;
}
```

Explanation:

The main function acts as the central hub of the Event Management System, presenting a menu-driven interface that allows users to perform tasks such as creating events, registering participants, and generating reports. It continuously loops to handle user inputs, calling the appropriate module functions until the user chooses to exit.

5.2 Functions Description

- `createEvent(int n):`
Purpose: Adds a participant and allows registration to specific events.

Code:

```
printf("\nEnter event name: ");
scanf("%s", events[eventCount].name);

printf("Enter event date (YYYY-MM-DD): "); // Simple date validation (not very robust)
scanf("%s", events[eventCount].date);
if (strlen(events[eventCount].date) != 10)
{
    printf("Invalid date format! Event creation aborted.\n\n");
    return;
}

printf("Enter event location: ");
scanf("%s", events[eventCount].location);

printf("Enter event description: ");
scanf("%s", events[eventCount].description);

printf("Enter max event participant: ");
scanf("%d", &events[eventCount].maxcount);

events[eventCount].pplcount=0;
eventCount++;

printf("\nEvent created successfully!\n\n");
```

Sample Output:

```
Enter your choice: 1
Enter event name: Winter Fest
Enter event date (YYYY-MM-DD): 2024-12-15
Enter event location: East West University
Enter event description: Celebrating the season of joy
Enter max event participant: 500
Event created successfully!
```

● updateEvent():

Purpose: Enables modification of event details (name, date, location, description)

o Code:

```
void updateEvent() //function to update an existing event
{
    int index;
    for (int i = 0; i < eventCount; i++)
    {
        printf("\n%d. Event : %s", i+1, events[i].name);
    }
    printf("\nEnter the index of the event to update (1 to %d): ", eventCount);
    scanf("%d", &index);

    index--; //index is declared as 1 but it is stored on the array index number 0, hence the decrement
    int change;

    if (index >= 0 && index < eventCount)
    {
        printf("What do you want to update : \n1. Event name.\n2. Event date.\n3. Event location.\n4. Event description.\nEnter : ");
        scanf("%d", &change);
        switch (change)
        {
            case 1 :
            {
                printf("Enter new event name: ");
                scanf("%s", events[index].name);
            }
            break;
            case 2 :
            {
                printf("Enter new event date: ");
                scanf("%s", events[index].date);
            }
            break;
            case 3 :
            {
                printf("Enter new event location: ");
                scanf("%s", events[index].location);
            }
            break;
            case 4 :
            {
                printf("Enter new event description: ");
                scanf("%s", events[index].description);
            }
            break;
            default:
                printf("category not found.");
                break;
        }
        printf("Event updated successfully!\n\n");
    }
    else
    {
        printf("Invalid event index!\n\n");
    }
}
```

```
Enter your choice: 2
=====
1. Event : Winter Fest
2. Event : Sports Fest
Enter the index of the event to update (1 to 2): 1
What do you want to update :
1. Event name.
2. Event date.
3. Event location.
4. Event description.
Enter :|
```


- `deleteEvent()`:

Purpose: Removes an event from the system.

Code:

```
void deleteEvent() //function to delete an existing event
{
    int index;
    for (int i = 0; i < eventCount; i++)
    {
        printf("\n%d. Event : %s", i+1, events[i].name);
    }
    printf("\nEnter the index of the event to delete (1 to %d): ", eventCount);
    scanf("%d", &index);

    index--;

    if (index >= 0 && index < eventCount)
    {
        for (int i = index; i < eventCount - 1; i++)
        {
            events[i] = events[i + 1];
        }
        eventCount--;
        printf("\nEvent deleted successfully!\n\n");
    }
    else
    {
        printf("Invalid event index!\n\n");
    }
}
```

Sample Output:

```
Enter your choice: 3
-----
1. Event : Winter Fest
2. Event : Pharma Fest
Enter the index of the event to delete (1 to 2): 1
Event deleted successfully!
```

- `registerParticipant(int n):`

- Purpose : Adds a participant and allows registration to specific events.
- Code:

```
void registerParticipant(int n) // Function to register a participant for an event
{
    if (eventCount == 0) // proof checking
    {
        printf("\nNo events available for registration.\n\n");
        return;
    }

    char name[100];
    printf("\nEnter participant name: ");
    scanf("%s", name); // takes input from the user until the new line command

    // checking if participant already exists
    int participantIndex = -1;
    for (int i = 0; i < participantCount; i++)
    {
        if (strcmp(participants[i].name, name) == 0)
        {
            participantIndex = i;
            break;
        }
    }

    if (participantIndex == -1) // New participant
    {
        if (participantCount >= 100)
        {
            printf("Participant limit reached! Cannot register more participants.\n\n");
            return;
        }

        // if the limit doesn't exceeds
        participantIndex = participantCount;
        strcpy(participants[participantIndex].name, name); // Initially participant index here is 0
        printf("Enter participant contact: ");
        scanf("%s", participants[participantIndex].contact);
        participants[participantIndex].eventCount = 0;
    }

    if (participants[participantIndex].eventCount >= n)
    {
        printf("Participant has reached the maximum number of events they can register for!\n\n");
        return;
    }

    for (int i = 0; i < eventCount; i++)
    {
        printf("\n%d. Event : %s", i+1, events[i].name);
    }

    printf("\nSelect event index (1 to %d): ", eventCount);
    int eventIndex;
    scanf("%d", &eventIndex);
    eventIndex--;

    if (eventIndex < 0 || eventIndex >= eventCount)
    {
        printf("Invalid event index! Registration aborted.\n\n");
        return;
    }

    events[eventIndex].ppcount++;
    if (events[eventIndex].maxcount < events[eventIndex].ppcount)
    {
        printf("Maximum participant registered for this event! Registration terminated.\n\n");
        return;
    }

    participants[participantIndex].eventList[participants[participantIndex].eventCount] = eventIndex;
    participants[participantIndex].eventCount++;
    printf("Participant '%s' registered for event '%s' successfully!\n\n", participants[participantIndex].name, events[eventIndex].name);
    participantCount++;
}
```

Sample Output:

```
Enter your choice: 4
-----

Enter participant name: Shahriar Hassan Mojumder
Enter participant contact: 01854-415147

1. Event : Pharma Fest
2. Event : Winter Fest
Select event index (1 to 2): 1
Participant 'Shahriar Hassan Mojumder' registered for event 'Pharma Fest' successfully!
```

● searchParticipantByName():

Purpose : Finds participant details by their name.

Code:

```
void searchParticipantByName() // Function to search for a participant by name
{
    char name[50];
    printf("\nEnter participant name to search: ");
    scanf("%s", name);
    int found = 0; // initialized as not found
    for (int i = 0; i < participantCount; i++)
    {
        if (strcmp(participants[i].name, name) == 0)
        {
            printf("\nParticipant Found:\nName: %s\nContact: %s\nRegistered Events:\n", participants[i].name, participants[i].contact);
            for (int j = 0; j < participants[i].eventCount; j++)
            {
                printf("- %s\n", events[participants[i].eventList[j]].name);
            }
            printf("\n");
            found = 1;
        }
    }
    if (found == 0)
    {
        printf("\nParticipant not found!\n\n");
    }
}
```

Sample Output:

```
Enter your choice: 5
-----

Enter participant name to search: Shahriar Hassan Mojumder

Participant Found:
Name: Shahriar Hassan Mojumder
Contact: 01854-415147
Registered Events:
- Pharma Fest
```

- `searchParticipantByEvent()`:

Purpose : Lists all participants registered for a particular event.

Code:

```
void searchParticipantByEvent(char eventName[]) {
    char eventNames[50];
    printf("\nEnter event name to search participants: ");
    scanf("%s", eventNames);
    int eventIndex = -1; //initialized as not found
    for (int i = 0; i < eventCount; i++)
    {
        if (strcmp(events[i].name, eventNames) == 0)
        {
            eventIndex = i;
            break;
        }
    }
    if (eventIndex == -1)
    {
        printf("\nEvent not found!\n\n");
        return;
    }
    printf("\nParticipants for event '%s':\n", eventNames);
    int found = 0;
    for (int i = 0; i < participantCount; i++)
    {
        for (int j = 0; j < participants[i].eventCount; j++)
        {
            if (participants[i].eventList[j] == eventIndex)
            {
                printf("Name: %s, Contact: %s\n", participants[i].name, participants[i].contact);
                found = 1;
            }
        }
    }
    if (found == 0)
    {
        printf("No participants registered for this event!\n\n");
    }
}
```

Sample Output:

```
Enter your choice: 6
-----

Enter event name to search participants: Pharma Fest

Participants for event 'Pharma Fest':
Name: Shahriar Hassan Mojumder, Contact: 01854-415147
```

- `generateReport()`:

Purpose: Outputs a list of all events, their details, and the number of registered participants

Code:

```

void generateReport(i) // function to generate report for ,showing the event names and the number of participants registered in there
{
    printf("\nEvent Report:\n");
    printf("-----\n");
    for (int i = 0; i < eventCount; i++)
    {
        int count = 0;
        for (int j = 0; j < participantCount; j++)
        {
            for (int k = 0; k < participants[j].eventCount; k++)
            {
                if (participants[j].eventList[k] == i)
                {
                    count++;
                }
            }
        }
        printf("\nEvent: %s\nEvent Date :%s\nEvent location :%s \nParticipants: %d\n\n", events[i].name, events[i].date, events[i].location, count);
    }
}

```

Sample Output:

```

Enter your choice: 7
-----

Event Report:
-----

Event: Pharma Fest
Event Date :2024-12-14
Event location :East West University
Participants: 1

Event: Winter Fest
Event Date :2024-12-12
Event location :East West University
Participants: 0

```

6. Testing and Results

6.1 Test Cases

Test Input	Expected Output	Actual Output
Event and it's data	Event added to to an array as an array element	Event registered
Participant's and their data	Participant info added to an array as an array element	Participant registered
Updating or deleting event	Manipulating array elements	Event list updated

Search of participant	Finding participant through comparing array elements string input	Participant found by their name or the event they've opted to register
Generate Report	Generating report by counting the total event , total participant and the event count of their registration	A well organized report ,helping to manage the event management system

6.2 Results and Screenshots

```

Enter the number of the events you want to manage by this code :2
===== EVENT MANAGEMENT SYSTEM =====

1. Create Event
2. Update Event
3. Delete Event
4. Register Participant
5. Search Participant by Name
6. Search Participant by Event
7. Generate Report
8. Exit

Enter your choice: 1
_____

Enter event name: Winter Fest
Enter event date (YYYY-MM-DD): 2024-12-12
Enter event location: East West University
Enter event description: Welcoming the season of joy and fun
Enter max event participant: 500

Event created successfully!

1. Create Event
2. Update Event
3. Delete Event
4. Register Participant
5. Search Participant by Name
6. Search Participant by Event
7. Generate Report
8. Exit

```

```

Enter your choice: 1
_____

Enter event name: Pharma fest
Enter event date (YYYY-MM-DD): 2024-12-29
Enter event location: East West University
Enter event description: A fest for all the people
Enter max event participant: 500

Event created successfully!

1. Create Event
2. Update Event
3. Delete Event
4. Register Participant
5. Search Participant by Name
6. Search Participant by Event
7. Generate Report
8. Exit

```

```
Enter your choice: 2
1. Event : Winter Fest
2. Event : Pharma fest
Enter the index of the event to update (1 to 2): 2
What do you want to update :
1. Event name.
2. Event date.
3. Event location.
4. Event description.
Enter :1
Enter new event name: Pharmacy Fest
Event updated successfully!

1. Create Event
2. Update Event
3. Delete Event
4. Register Participant
5. Search Participant by Name
6. Search Participant by Event
7. Generate Report
8. Exit
```

Enter your choice: 4 ----- Enter participant name: Sabiha Khanam Enter participant contact: 01xxxxxxx 1. Event : Winter Fest 2. Event : Pharmacy Fest Select event index (1 to 2): 1 Participant 'Sabiha Khanam' registered for event 'Winter Fest' successfully! 1. Create Event 2. Update Event 3. Delete Event 4. Register Participant 5. Search Participant by Name 6. Search Participant by Event 7. Generate Report 8. Exit Enter your choice: 4 ----- Enter participant name: Noor Maisha Enter participant contact: 01xxxxxxx 1. Event : Winter Fest 2. Event : Pharmacy Fest Select event index (1 to 2): 2 Participant 'Noor Maisha' registered for event 'Pharmacy Fest' successfully!	1. Create Event 2. Update Event 3. Delete Event 4. Register Participant 5. Search Participant by Name 6. Search Participant by Event 7. Generate Report 8. Exit Enter your choice: 5 ----- Enter participant name to search: Noor Maisha Participant Found: Name: Noor Maisha Contact: 01xxxxxxxxx Registered Events: - Pharmacy Fest 1. Create Event 2. Update Event 3. Delete Event 4. Register Participant 5. Search Participant by Name 6. Search Participant by Event 7. Generate Report 8. Exit Enter your choice: 6 ----- Enter event name to search participants: Winter Fest
Enter event name to search participants: Winter Fest Participants for event 'Winter Fest': Name: Sabiha Khanam, Contact: 01xxxxxxxxx 1. Create Event 2. Update Event 3. Delete Event 4. Register Participant 5. Search Participant by Name 6. Search Participant by Event 7. Generate Report 8. Exit Enter your choice: 7 ----- Event Report: ----- Event: Winter Fest Event Date :2024-12-12 Event location :East West University Participants: 1 Event: Pharmacy Fest Event Date :2024-12-29 Event location :East West University Participants: 1	1. Create Event 2. Update Event 3. Delete Event 4. Register Participant 5. Search Participant by Name 6. Search Participant by Event 7. Generate Report 8. Exit Enter your choice: 8 ----- Exiting...

7. Conclusion

Summary:

This Event Management System is a comprehensive program designed to efficiently handle the creation, modification, and deletion of events, along with participant registration and tracking. It offers functionality to generate detailed reports and search for participants based on their registrations. With a user-friendly interface and robust modular structure, it streamlines event management tasks for both organizers and participants.

Knowledge Gained:

- Enhanced understanding of modular programming and the importance of structured code design for clarity and re-usability.
- Improved problem-solving skills by implementing features such as event creation, participant registration, and search functionalities.
- **Group-Collaboration:**

Effective teamwork led to better task distribution and idea generation.

Challenges Faced:

- **Data Validation:** Ensuring user inputs, such as dates and names, adhered to required formats and constraints.
- **User Interface Challenges:** Making the menu-driven interface intuitive while providing clear error messages and feedback.

Opportunities for Improvement:

- Adding a graphical interface.
- Implementing a database for better scalability.
- Introducing user authentication for manager.

Remarks:

The project was a valuable learning experience, enhancing both technical and collaboration skills.

9. References

- ✓ Class Lecture,
- ✓ Teach Yourself C - Herbert Schildt
- ✓ Open source
- ✓ W3School
- ✓ Geeksforgeek